

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

Code of Conduct:

I R.SHOBANA SHRI(1921260053) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3

3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3

9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3

16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Redesigning Testing for a Banking Application

Testing as an Engineering Activity:

Testing is treated as a planned, measurable and repeatable process (like designing a bridge), not a last-minute activity done by intuition.

1. Systematic Test Design

- Derive test cases from requirements (e.g., "transfer limit ₹1,00,000/day"), with traceability from requirement → test case → defect.
- Follow a defined lifecycle: plan → design → execute → report → improve.
- Define entry/exit criteria, e.g., release only when 100% of critical cases pass and no high-severity defects remain open.

2. Tools and Standards

- **Tools:** Selenium/Postman for automation, JIRA for defect tracking, Jenkins for CI regression runs, JMeter for load testing.
- **Standards:** IEEE 829 test documentation, ISTQB terminology, and a common defect severity/priority template. Compliance checks such as PCI-DSS apply for banking security.

3. Techniques Replacing Ad-hoc Testing

- **Boundary Value Analysis:** transfer limits and minimum balance.
- **Equivalence Partitioning:** valid/invalid amounts and account types.
- **Decision Table:** loan approval rules.
- **State Transition:** account states (active → frozen → closed).
- **Regression + Risk-based testing:** high priority for payments and login.

4. Practical Example: Fund Transfer (limit ₹50,000)

Technique	Test Inputs	Expected
BVA	49,999 / 50,000 / 50,001	Pass / Pass / Reject
EP	Negative, zero, text, valid	Reject, Reject, Reject, Accept
State Transition	Transfer from frozen account	Blocked

Ad-hoc testers typically try only a "normal" amount like ₹5,000 and miss the 50,001 off-by-one defect; BVA catches it directly.

5. How Quality Improves

- **Defect detection:** boundary and rule-based defects are found early, so fixing is cheaper.
- **Coverage:** measurable (requirement and code coverage) instead of guesswork.
- **Repeatability:** automated regression prevents old bugs returning after each release.
- **Metrics:** defect density and defect removal efficiency show progress, enabling continuous improvement.

Conclusion:

Applying engineering discipline (planned design, standard tools and documentation, and proven techniques) turns random testing into a controlled process, reducing production failures in the banking system.

- 2. An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.**

Structured Testing Process for an ATM System

Approach:

Testing is embedded in every SDLC phase as a defined, repeatable process (the V-model idea: each development phase has a matching test activity), instead of being done once at the end.

Stages Across the SDLC

SDLC Phase	Test Activity	Relevance to Quality
Requirements	Review requirements for ambiguity, define testable criteria	Removes defects at the cheapest point
Design	Prepare test plan, test strategy, risk analysis, and design test cases	Gives early coverage and a clear scope
Coding	Unit testing, code review, static analysis	Catches logic faults close to their source
Integration	Interface testing between ATM, card reader, bank server, and cash dispenser	Finds communication and data-flow defects
System / Acceptance	Functional, security, performance, and UAT against requirements	Confirms the whole system meets business needs
Maintenance	Regression testing after each fix or release	Prevents old defects from returning

Applying to ATM Withdrawal

Requirement: daily withdrawal limit ₹20,000, multiples of ₹100, balance must cover the amount.

- **Requirements stage:** Clarify the unclear points, e.g., what happens when the ATM holds less cash than requested?
- **Test design:** Boundary values 19,900 / 20,000 / 20,100, invalid amounts (150, 0, -500), insufficient balance, and wrong PIN three times (card block).
- **Unit/Integration:** Test the balance-check module, then verify that the account is debited exactly once after the cash is dispensed.
- **System testing:** Network failure mid-transaction (the account must not be debited if no cash is dispensed), power loss, and concurrent withdrawals from the same account.
- **Regression:** Re-run the withdrawal suite after any change to the fee or limit logic.

How Consistency Reduces Defects

- The same documented procedure gives the same results regardless of who tests, so the "inconsistent results" problem disappears.
- Entry/exit criteria stop untested builds from moving forward.
- Defect logs and traceability link each failure to its requirement and phase, so root causes (e.g., ambiguous limits) are fixed at the source.
- Early detection cuts rework cost, and reuse of the test suite saves effort every cycle.

Expected Outcomes

- Fewer production failures such as double debits or wrong dispense amounts
- Higher defect removal efficiency, with most bugs found before release
- Predictable, repeatable results and measurable coverage
- Improved customer trust, security, and regulatory compliance

Conclusion:

A phase-wise, documented testing process turns ATM testing from guesswork into a controlled activity, giving consistent behavior and reliable withdrawals.

- 3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.**

Error, Defect and Failure in a Web Application

Basic Definitions (ISTQB/IEEE)

- **Error (mistake):** A human action that produces an incorrect result, made by a developer, analyst or designer.
- **Defect (bug/fault):** The flaw in code or documentation that results from an error.
- **Failure:** The visible deviation of actual output from expected output when the defect is executed.

Relationship:

Error → Defect → Failure. Not every defect causes a failure; it shows only when the faulty code path runs.

Applying to Login and Transaction

Scenario	Error	Defect	Failure
Login	Developer misreads the requirement "lock after 3 wrong attempts"	Code uses attempts > 3 instead of >= 3	The account locks after the 4th attempt, so the user is not blocked at the 3rd
Transaction	Developer ignores the session-timeout case	No check that the session is valid before posting	A transaction is processed with an expired session, or the amount is debited twice on refresh

The mismatch between expected and actual output is the failure the tester observes; the tester then traces it back to the defect, and the root cause is the error.

Effect of Misunderstanding These Terms

- Testers report symptoms ("login broken") without isolating the underlying defect, so developers waste time reproducing it.
- Treating a failure as the root cause leads to patching the symptom (e.g., hiding the error message) while the defect remains.
- Defect metrics become unreliable, because failures, defects and enhancement requests are counted together.
- Root-cause analysis of the human error (unclear requirement, missing review) is skipped, so the same errors recur in later releases.

Improving Defect Reporting Clarity

A good report separates observation from analysis and includes:

- **Unique ID and title:** "Account not locked after 3rd invalid password."
- **Steps to reproduce:** Enter a valid user ID, then 3 wrong passwords.
- **Expected vs actual result:** Account locked vs. 4th attempt still allowed.
- **Environment:** Browser, OS, build version.
- **Severity and priority:** High severity, since it is a security risk.
- **Evidence:** Screenshot, logs, network trace.

Example:

"Login fails" is vague. "Build 2.3, Chrome 118: after 3 wrong passwords, the 4th attempt still reaches the password check; expected: account locked" lets a developer locate the defect at once.

Conclusion:

Clear use of error, defect and failure lets the tester describe what was seen (failure), where it lies (defect) and why it arose (error). This precision speeds up debugging, gives accurate metrics and prevents recurring faults in login and transaction modules.

- 4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.**

Applying Testing Principles to Defect-Prone Modules

Principle in Focus: Defect Clustering

A small number of modules usually hold most of the defects (the Pareto idea: roughly 80% of defects come from 20% of modules). Instead of spreading effort evenly, the team directs testing where the defects actually are.

How Principles Guide Test Planning

Principle	Planning Action
Defect clustering	Use defect history and complexity metrics to identify hot-spot modules and give them deeper testing
Exhaustive testing is impossible	Select test cases by risk instead of trying to test every input
Early testing	Review and test the risky modules from the requirements and design stages
Pesticide paradox	Revise and add test cases regularly, since the same tests stop finding new bugs

Principle	Planning Action
Testing is context dependent	Plan stricter security and accuracy testing for payment modules than for a help page
Absence-of-errors fallacy	Check that the product meets user needs, not only that few bugs remain

Prioritizing Test Cases

1. **Collect data:** Count defects per module from past releases and the current cycle.
2. **Assess risk:** Score each module as likelihood of failure × business impact.
3. **Rank:** Run high-risk, high-defect modules first; run low-risk modules with lighter checks.
4. **Select by value:** Within a module, run test cases linked to critical requirements and past failures first, followed by boundary and negative cases.
5. Re-evaluate after every cycle, because clusters shift as fixes are made.

Real-Time Example: Online Shopping Application

Defect logs show 60 defects in total: 35 in Payment/Checkout, 15 in Cart, and the remaining 10 spread over Search, Profile and Wishlist.

- Payment and Cart (83% of defects) get extra test cases covering coupon plus tax combinations, failed payment retries, and duplicate-order checks, along with code review and additional regression runs.
- Search and Wishlist get basic smoke tests only.
- Developers also refactor the checkout module, since the cluster usually points to a design or complexity problem.

How Efficiency Improves

- Testing time goes where defects are most probable, so more bugs are found per hour.
- Delays shrink because critical defects surface early, before the release deadline.
- Resources are used according to risk, avoiding wasted effort on stable modules.
- Root causes of the cluster (complex logic, an unclear requirement) can be fixed, cutting future defects.

Conclusion:

Defect clustering, supported by risk-based prioritization and the other principles, turns random effort into focused testing. The team finds more defects earlier, reduces delays and delivers higher quality with the same resources.

5. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Using a Defect Repository to Stop Recurring Defects

How Defects Originate

Defects enter at every SDLC phase, usually from a human error:

- **Requirements:** ambiguous, missing or conflicting requirements
- **Design:** wrong logic, weak data model, poor interface design
- **Coding:** off-by-one errors, unhandled nulls, wrong conditions
- **Testing/Environment:** incorrect test data, wrong configuration
- **Maintenance:** a fix or change that breaks existing behavior

Defect Classification (Categories)

Basis	Categories	Example
Origin / phase	Requirement, design, code, test, deployment	Wrong tax rule in the spec
Type	Logic, interface, data, performance, security, usability	Calculation error in interest
Severity	Critical, major, minor, trivial	Wrong balance shown = critical
Priority	High, medium, low	Fix before release or later
Module / status	Payment, login, reports; open, fixed, reopened	Recorded for trend analysis

Each defect should also record root cause and phase detected vs. phase injected, because these fields reveal where the process is failing.

Using Repository Data

Test design:

- Defect history shows which modules and defect types occur most, so test cases target them (defect clustering).
- Each fixed defect becomes a permanent regression test case.
- Past boundary and data defects suggest new negative and boundary tests.

Prevention:

- Root-cause analysis (Pareto charts, fishbone diagram) shows the main sources, e.g., 40% of defects trace to unclear requirements.
- Process fixes, such as requirement reviews or coding checklists, then target that cause.

Example:

A repository shows that "rounding error in interest calculation" was logged in three releases. Analysis finds a missing rule for decimal handling in the specification. The team clarifies the rule, adds boundary tests for rounding, adds the cases to automated regression, and adds a checklist item to code review. The defect stops recurring.

Improvements to Avoid Recurrence

- Make classification fields mandatory and standardized (using a scheme like IEEE 1044) so data is reliable.
- Search the repository for duplicates before closing or raising a defect.
- Hold regular defect review meetings and root-cause analysis for major defects.
- Link defects to requirements, test cases and code changes for traceability.
- **Track metrics:** defect density, recurrence rate, reopen rate and defect removal efficiency.
- Share lessons learned with developers and update checklists and test suites each release.

Expected Quality Improvements

- Fewer repeated defects and a lower reopen rate
- Earlier detection, since test cases are driven by real defect history
- Reduced rework cost and fewer release delays
- Measurable process improvement and a more stable product

Conclusion:

A defect repository becomes useful when defects are classified consistently and analyzed for root causes. Feeding that knowledge back into test design and development practice moves the team from fixing the same bugs again to preventing them.

- 6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.**

Diagnosing Crashes in Mobile Banking Fund Transfer

Principles Applied to Find the Root Cause

Principle	Application to the crash
Testing shows presence of defects	Crash logs prove defects exist, so reproduce each crash rather than assume it is random
Defect clustering	Check crash reports: if most come from the transfer module, concentrate testing there
Early testing	Review the transfer requirements and API contracts for gaps (timeouts, duplicate requests)
Exhaustive testing is impossible	Choose inputs and devices by risk, not by trying every combination
Pesticide paradox	Refresh old test cases, since the same suite no longer exposes the crash
Context dependence	Mobile needs tests for network switches, interruptions and device variety

Root Cause Analysis Steps

- 1. Collect evidence:** crash logs, stack traces, device/OS versions, app build and time of failure.
- 2. Reproduce:** repeat the exact steps (amount, payee, network state) until the failure is consistent.
- 3. Isolate:** narrow it down by changing one factor at a time (UI, API, database, network).
- 4. Trace:** link the failure to the defect and then to the originating error (e.g., a missing requirement for timeouts).
- 5. Fix and retest:** confirm the fix, then run regression.

Illustrative Examples:

Observed crash	Likely defect	Test that exposes it
Crash when tapping "Transfer" twice	No duplicate-request handling	Rapid double-tap test
Crash on switching from Wi-Fi to 4G mid-transfer	Unhandled network exception	Network interruption test
Crash with a ₹0 or very large amount	Missing input validation / overflow	Boundary value analysis
Crash on low-memory or older Android	Memory leak, device incompatibility	Device matrix and stress test

Observed crash	Likely defect	Test that exposes it
Crash when a call arrives during OTP	State lost during interruption	Interrupt testing

How Systematic Testing Reduces Such Issues

- Test cases are designed from requirements using BVA, equivalence partitioning and state transition, so coverage is deliberate.
- Failures are found in a controlled environment before users see them.
- Every crash gets a logged defect with steps, logs and severity, which supports traceability and root-cause analysis.
- Automated regression ensures old crashes do not return.

Improvements in Testing Practices

- **Automation:** Appium/Espresso for regression on every build via CI.
- **Device coverage:** cloud device farms to test across OS versions and screen sizes.
- **Non-functional tests:** load testing of the transfer API (JMeter), performance, memory and battery profiling.
- **Resilience tests:** network loss, timeouts, server errors, app kill and resume.
- **Monitoring:** crash analytics (e.g., Firebase Crashlytics) to feed real failures back into test design.
- **Security and idempotency checks:** guarantee one debit per transaction even when retried.
- **Reviews and exit criteria:** release only when no critical crash remains open.

Conclusion:

Applying testing principles and a structured root-cause process turns random crashes into traceable defects. Systematic design, broader device and resilience testing, and automation reduce failures in fund transfers and improve reliability and customer trust.

- 7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.**

Testing vs Debugging in Defect Resolution

Core Concepts

- **Testing:** A planned activity that executes the software to expose failures and verify that it meets requirements. It is usually done by testers, and it can be automated.
- **Debugging:** The activity of finding the exact location and cause of a known failure and fixing it. It is done by developers and is a diagnostic, code-level task.

In short: testing *reveals* that something is wrong; debugging finds *why* and corrects it.

Differences

Aspect	Testing	Debugging
Purpose	Detect failures, validate quality	Locate the defect and remove it
Performed by	Tester (can be independent)	Developer
Trigger	Planned, scheduled in the test plan	Starts after a failure is reported
Output	Test results, defect reports	Code fix, root-cause finding
Predictability	Can be planned and automated	Time needed is hard to predict
Knowledge needed	Requirements and test techniques	Code and design internals

Roles in Software Quality

- Testing provides evidence of quality: coverage, pass/fail status and defect trends.
- Debugging improves quality by actually removing the defects testing finds.
- Retesting after debugging confirms the fix, and regression testing confirms nothing else broke.

Effects of Improper Usage

Misuse	Impact	Example
Developers "test" only while debugging	Only the known failure is checked, so new defects go unfound	A fix for login lockout is checked, but the password-reset flow stays untested
Testers try to fix code or guess causes	Weak reports, and the tester's time is spent on the wrong activity	A report says "the bug is in the DB query" with no evidence or steps
No retest or regression after a fix	Defects reappear or fixes introduce new faults	A tax fix breaks the invoice total

Misuse	Impact	Example
Treating debugging as a substitute for testing	Defect detection depends on chance, with no coverage measure	The release ships with unexercised modules

Structured Approach to Integrate Both

1. Plan and design tests from requirements (BVA, equivalence partitioning) and execute them.
2. Log the failure with steps, expected vs actual result, environment and severity.
3. Debug: the developer reproduces the failure, isolates the faulty code (logs, breakpoints, code tracing), finds the root cause and fixes it.
4. Retest the fix, handled by the tester.
5. Run regression on related modules, preferably automated in CI.
6. Update the repository and test suite: record the root cause and add a new test case for the defect.
7. Review at the end of the cycle to improve requirements, coding checklists and test design.

Example:

A fund transfer shows a wrong balance. The tester's boundary test exposes the failure and reports it. The developer debugs and finds a rounding defect in the calculation. The tester retests, runs the regression suite, and adds a permanent rounding test.

Conclusion:

Testing and debugging are separate but complementary. Testing finds failures systematically, debugging removes their causes, and retesting closes the loop. Keeping their roles distinct and linked gives faster fixes, better defect detection and higher product quality.

8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Cross-Browser Testing Strategy for an E-commerce Platform

Testing Principles Applied

Principle	Application
Exhaustive testing is impossible	Cannot test every browser, version and device, so build a priority matrix from analytics data
Defect clustering	Most inconsistencies sit in a few areas (checkout forms, payment widgets, CSS layout), so test these deeper
Early testing	Check design and front-end code for browser-specific features during development, not before release
Pesticide paradox	Refresh test cases as browsers auto-update and new versions behave differently
Context dependence	An e-commerce site needs strict checkout and payment checks; a blog page needs only layout checks
Absence-of-errors fallacy	Passing on one browser does not mean the product works for users on others

How Defects Originate

- **Different rendering engines:** Blink (Chrome/Edge), WebKit (Safari), Gecko (Firefox) interpret CSS and HTML differently.
- **Unsupported features:** newer JavaScript or CSS (e.g., a grid or API missing in older Safari) used without fallbacks.
- Missing vendor prefixes or polyfills for older versions.
- Different form, cookie and storage handling, e.g., autofill or third-party cookie blocking breaking the login or cart.
- **Requirement gap:** the supported browser list was never defined (an error at the requirements stage).
- **Device variation:** screen size, resolution, touch vs. mouse input.

Strategy

1. **Define scope:** use analytics to pick the top browsers, versions and devices (e.g., Chrome, Safari, Edge, Firefox, mobile Chrome/Safari) and record them as a support matrix.
2. **Prioritize journeys:** search → product page → cart → checkout → payment → order confirmation.

3. **Design tests from requirements:** functional, UI/layout, responsive, performance and compatibility checks.
4. Execute manually for exploratory UI review and automatically for repeated regression.
5. Log defects with browser, version, OS, screenshot and steps, then retest and run regression.

Example

Checkout: the "Pay Now" button works in Chrome but is hidden in Safari because of an unsupported CSS property, and the date field in Firefox rejects the format Chrome accepts. Test cases run on the matrix expose both; the fix adds a fallback style and a standard date format, and the cases are added to the regression suite.

How Systematic Testing Improves Reliability

- A defined matrix gives measurable coverage instead of testing only the browser the developer uses.
- Automation repeats the same checks on every build, so regressions are caught immediately.
- Logged browser-specific defects reveal patterns, which leads to fixes such as CSS resets and feature detection.
- Users see consistent behavior, lowering cart abandonment and support calls.

Tools and Techniques

Category	Examples
Cross-browser cloud platforms	BrowserStack, LambdaTest, Sauce Labs
Automation	Selenium WebDriver/Grid, Playwright, Cypress
Visual regression	Percy, Applitools
Compatibility checks	Can I use, linters and Babel/Autoprefixer for fallbacks
CI integration	Jenkins/GitHub Actions to run suites on each build
Techniques	Equivalence partitioning on browser/OS combinations, pairwise testing to reduce combinations, responsive testing, exploratory sessions

Conclusion:

Applying testing principles turns ad-hoc browser checks into a risk-based, measurable strategy. Prioritized coverage, automation and visual comparison find browser-specific defects early, giving the platform consistent and reliable behavior for every user.

- 9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.**

Classifying and Preventing Logical Defects in a Payroll System

Defect Classification of Salary Errors

Basis	Category	Payroll Example
Origin (phase)	Requirement	Overtime rule never specified for public holidays
	Design	Tax slab logic designed without handling mid-year joiners
	Coding	Wrong operator or condition in the formula
Type	Algorithm/logic	Net pay = gross + deductions, instead of gross – deductions
	Computation/data	Rounding errors, integer division truncating decimals
	Boundary/condition	Tax slab applied at $\geq 5,00,000$ instead of $> 5,00,000$
	Interface/data flow	Attendance module sends wrong days-worked value
Severity	Critical / Major / Minor	Wrong net pay = critical; wrong payslip label = minor
Priority	High / Medium / Low	Fix before the month-end payroll run

How Understanding Defect Classes Improves Testing

- **Each class points to a matching technique:** boundary defects → BVA, rule combinations → decision tables, calculation defects → equivalence partitioning and formula-based oracles.
- Defect counts per class show where the process is weak, so effort goes to the highest-risk areas (defect clustering).
- Classification by origin shows whether the problem lies in requirements, design or code, so the correct fix is chosen.
- Severity and priority let the tester order retesting around the payroll deadline.

Illustrative Example

Defect: Employee with ₹49,999 basic pay gets the wrong tax, while ₹50,001 is correct.

- **Class:** boundary/logic defect, coding origin, critical severity.
- **Root cause:** the condition $>$ was used where $\geq$ was needed.
- **Test that finds it:** BVA at 49,999 / 50,000 / 50,001 with precomputed expected salaries.

Prevention in Future Releases

- Clarify and review the pay rules (tax slabs, overtime, leave deduction) before coding.
- Use code reviews with a checklist covering conditions, rounding and unit handling.
- Add unit tests for every formula, run automatically on each commit.
- Turn every fixed defect into a permanent regression test.
- Perform root-cause analysis on the repository and update the checklist for each recurring class.

Improvements in Testing Strategy

- **Techniques:** BVA, decision tables for deduction rules, state transition for employee status (joined, on leave, resigned).
- **Independent oracle:** compare system output against a validated spreadsheet or the previous trusted version (parallel testing).
- **Data-driven automation:** run hundreds of salary scenarios from a data file via Selenium or JUnit.
- **Risk-based prioritization:** test tax, overtime and arrears modules first.
- **Regression and metrics:** track defect density and defect removal efficiency per release.

Conclusion:

Classifying payroll defects by origin, type and severity connects each failure to a suitable test technique and a prevention step. This structured approach finds logical errors earlier, stops repeat faults and gives accurate, trustworthy salaries.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Replacing Random Testing with Engineered Test Design

Testing as an Engineering Activity

Engineering means a planned, measurable and repeatable approach. Random execution depends on the tester's mood and memory; engineered testing starts from requirements, produces documented test cases, and judges results against defined criteria.

Redesigned Process

1. Analyze requirements and identify testable conditions.
2. **Plan:** scope, risks, environment, entry/exit criteria.
3. **Design test cases:** ID, objective, preconditions, input data, steps, expected result.
4. Review the cases with peers and developers.
5. Execute and log results against the expected outcomes.
6. **Report and measure:** coverage, pass/fail, defect density.
7. **Improve:** update the suite from the defects found.

Importance of Structured Test Case Design

- **Coverage:** each requirement maps to at least one test case (traceability), so gaps are visible.
- **Repeatability:** any tester can rerun the same case and get the same result.
- **Clear pass/fail:** the expected result is decided before execution, not guessed afterward.
- **Efficiency:** fewer redundant tests and a smaller, more valuable suite.
- **Reuse:** cases become the base for regression and automation.
- **Accountability:** the team can show what was tested and what was not.

How It Improves Defect Detection

Example: Online booking, "age must be 18 to 60".

Approach	Tests	Result
Random	Tries 25, 30, 40	All pass, so the defect is missed
Designed (BVA + EP)	17, 18, 60, 61, plus text and blank	Finds that 60 is wrongly rejected (<60 coded instead of <=60)

The random tests all sit inside the valid partition, so they never touch the boundary where the defect lives. Designed tests target error-prone points deliberately and also include negative cases that random testers skip.

Techniques to Enhance Effectiveness

Type	Technique	Use
Black-box	Equivalence partitioning	Reduce inputs to representative classes
	Boundary value analysis	Limits and thresholds
	Decision table	Combined business rules
	State transition	Workflows and statuses
	Use-case testing	End-to-end user scenarios
White-box	Statement/branch coverage	Ensure code paths are executed
Experience-based	Error guessing, exploratory testing	Add on top of designed tests, with a time-boxed charter
Support	Risk-based prioritization, pairwise testing	Focus effort and cut combinations

Conclusion:

Structured test design converts testing from chance into a controlled engineering process. Requirement-based cases, proven techniques and measurable coverage detect more defects, earlier and more reliably, than random execution.

11.A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Data Inconsistency in a Healthcare Application

Testing Principles Applied

Principle	Application
Testing shows presence of defects	Each mismatch (e.g., patient record differs between modules) is evidence of a defect that must be traced, not ignored
Defect clustering	Inconsistencies usually gather at integration points such as Registration → Billing → Pharmacy → Lab, so test these interfaces deeply
Early testing	Review the data model and interface specifications before coding to catch mapping gaps
Exhaustive testing is impossible	Prioritize critical data (patient ID, allergies, prescriptions, dosage) by risk to patient safety
Context dependence	Healthcare demands stricter accuracy, audit and privacy checks (HIPAA-style compliance) than a general app
Absence-of-errors fallacy	Modules passing individually does not prove the combined data is correct

How Defects Originate During Development

- **Requirements:** no single definition of shared data (e.g., "patient name" format or unit of weight differs per team).
- **Design:** duplicate data stores without a master source; no synchronization rule.
- **Coding:** field mapping errors, wrong data type or length, truncation, unit mismatch (kg vs lb, mg vs mcg).
- **Integration:** an API sends a changed schema, or a failed transaction updates one module but not the other.
- **Concurrency:** two users edit the same record and one update overwrites the other.
- **Maintenance:** a change in one module is not propagated to dependants.

Systematic Testing for Data Integrity

Test Type	Check	Example
Integration testing	Data passes correctly between modules	Allergy entered in Registration appears in Pharmacy
Data validation (BVA/EP)	Field length, type, range	Age 0, 120, negative, text
CRUD and consistency testing	Create/update/delete reflects everywhere	Updated address shows in Billing and Reports
Transaction testing	All-or-nothing updates (ACID)	A failed payment must not leave a "paid" status in the Records module
Concurrency testing	Simultaneous edits	Two nurses updating one chart
Migration/reconciliation	Source vs target comparison	Row counts and checksums after data load
Regression	Recheck after each change	Schema update does not break the lab feed

Example:

A patient's dosage is stored as 5 mg in Prescription but displayed as 50 mg in Pharmacy because of a decimal-mapping error. An integration test with precomputed expected values exposes it before release, which avoids a medication safety risk.

Improvements in Defect Tracking and Reporting

- Record module, interface, data field, build and environment in each defect so integration issues can be located quickly.
- Add patient-safety severity (critical when clinical data is affected) and set fix deadlines by severity.
- Include evidence: screenshots, database queries and logs, and the expected vs actual value in both modules.
- Classify by origin (requirement, design, code, integration) and record the root cause for trend analysis.
- Link each defect to its requirement, test case and code change (traceability), and add every fixed defect to the regression suite.
- Use tools such as JIRA with dashboards on defect density, reopen rate and aging defects.
- Keep patient data anonymized in reports to maintain privacy.

Conclusion

Applying testing principles focuses effort on risky integration points, while systematic checks on mapping, transactions and concurrency protect data integrity. Better classified and traceable defect reporting finds root causes sooner and prevents repeat inconsistencies, giving a safer and more reliable healthcare system.

12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Using a Defect Repository to Break the Cycle of Repeated Defects

Why Defects Keep Recurring

Testing finds defects, but the knowledge from them is not reused. Common causes are weak classification (everything logged as "bug"), no root-cause field, duplicates, and fixes that never become permanent tests. The repository then acts as a storage box instead of a learning tool.

Role of the Defect Repository

It is a central, searchable store of every defect with its details:

Field	Purpose
ID, title, module, build	Locate where the defect occurred
Type, severity, priority	Classify and order the effort
Phase injected vs detected	Show which stage lets defects through
Root cause, fix, status	Support prevention and reopen analysis
Linked requirement and test case	Traceability

How Repository Data Supports Test Design

- **Targeted cases:** modules with many past defects get deeper tests (defect clustering).
- **New test ideas:** past boundary, null-input or rounding defects suggest similar negative and boundary cases for other modules.
- **Regression suite:** each fixed defect becomes a permanent test case, so it cannot silently return.
- **Risk-based priority:** frequency × severity of past defects decides which tests run first.
- **Checklists:** common defect types feed review and test checklists.

Identifying Defect Patterns

1. Extract defects over several releases.
2. Group them by module, type, origin phase and root cause.
3. Analyze using a Pareto chart (the few categories behind most defects), trend charts per release, and a fishbone diagram for root causes.
4. Interpret the pattern and decide the corrective action.

Example:

Across four releases, 55 defects are logged for an e-commerce app. The repository shows 22 are in Coupon/Discount, 18 of those are "calculation and boundary" type, and most were injected at the requirements phase. The pattern tells us the discount rules were never clearly specified. The fix is not

just more testing: the team writes a decision table for coupon rules, adds boundary tests, reviews the requirement, and adds the cases to automated regression. Recurrence drops in the next release.

Improvements to Prevent Recurrence

- Make fields (type, root cause, phase injected) mandatory and standardized, for example following IEEE 1044.
- Search for duplicates and similar past defects before logging or closing one.
- Hold root-cause analysis for critical and repeated defects, and assign preventive actions with owners.
- Feed lessons into requirement reviews, coding checklists and the test suite every release.
- Track metrics: recurrence rate, reopen rate, defect density and defect removal efficiency.
- Use dashboards in JIRA or similar tools, and review them in release retrospectives.

Conclusion:

A defect repository is valuable only when its data is classified, analyzed and fed back into test design and development. Pattern analysis turns past failures into targeted tests and process fixes, which cuts repeated defects, lowers rework and improves release quality.

13.A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Closing Testing Gaps in a Web-Based Learning System

Testing Principles Applied to Find the Gaps

Principle	Gap Revealed
Exhaustive testing is impossible	Without risk analysis, peak-load scenarios (exam day, assignment deadlines, course registration) were never chosen for testing
Early testing	Performance and capacity requirements (e.g., "5,000 concurrent users") were never defined or reviewed at the requirements stage
Defect clustering	Failures concentrate in login, video streaming, quiz submission and file upload, which need deeper load coverage
Context dependence	A learning platform has sharp, predictable traffic spikes, so functional tests alone are not enough
Pesticide paradox	The same functional suite was reused each release and cannot expose concurrency or resource defects
Absence-of-errors fallacy	The system passed functional tests but still failed users when it mattered

Impact of Test Planning and Coverage on Performance

- Planning defines non-functional goals (response time under 3 seconds, 99.9% availability), the load profile, the production-like environment and exit criteria. Without it, performance testing is skipped or done on a tiny setup.
- Coverage must extend beyond functions to include load, stress, spike, endurance and scalability scenarios, plus infrastructure limits such as database connections, memory and bandwidth.
- Low coverage means bottlenecks (slow queries, thread exhaustion, session leaks) stay hidden until real users trigger them.

How Better Strategy Improves Reliability

Example:

On the day of an online exam, 4,000 students log in within five minutes and submit quizzes at the same time. The system slows down and then crashes.

- A load test at the expected peak would show response times rising after 1,500 users.
- A stress test would find the breaking point, and monitoring would reveal the cause, such as an unindexed quiz-submission query or exhausted connection pool.
- After tuning (indexing, caching, connection pool sizing), a retest confirms stability at 5,000 users with a safety margin.

The team finds the bottleneck before release, so failure is replaced by a measured, known capacity.

Enhancements to Avoid Future Failures

- Define NFRs (concurrent users, response time, throughput) in the requirements and trace them to tests.
- Performance testing tools: JMeter, Gatling or k6 for load, stress, spike and soak tests; run them in a production-like environment.
- Realistic workload models built from analytics of past peak days.
- Monitoring and profiling: APM tools (New Relic, Grafana, Prometheus) to locate bottlenecks.
- CI integration: automated performance checks on every major build, with baselines to detect degradation.
- Resilience measures: autoscaling, caching/CDN, queue-based submission, graceful degradation, and failover tests.
- Risk-based planning: test critical journeys (login, exam, submission) first, and update scenarios after each incident.
- Post-release feedback: use production logs and defects to refine test cases each cycle.

Conclusion:

Applying testing principles exposes that the failure came from missing non-functional planning and coverage rather than missing functional tests. Defined performance goals, realistic load scenarios, tooling and monitoring let the team find bottlenecks early, giving a learning system that stays reliable at peak usage.

14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Involving Testers Early in the SDLC

Role of the Tester Across the SDLC

SDLC Phase	Tester's Role
Requirements	Review requirements for ambiguity, gaps and testability; define acceptance criteria
Design	Review design and interfaces; prepare the test strategy and plan; identify risks
Coding	Write test cases and test data in parallel; support unit and code-review checks; set up the environment
Testing	Execute tests, log defects, retest and run regression
Deployment	Smoke test the release, then verify in production
Maintenance	Regression and impact analysis for every change

This follows the "early testing" principle: testing activities start as soon as the project starts, not after code is complete.

How Early Involvement Impacts Quality

- **Cheaper fixes:** a defect found in requirements costs far less than one found after release, since no code, rework or redeployment is needed.
- **Better requirements:** testers ask "how will I verify this?", which exposes vague or conflicting statements.
- **Prevention over detection:** defects are stopped from being injected, not just found later.
- **More test time:** design of test cases happens in parallel with development, so execution begins sooner.
- **Shared quality ownership:** developers and testers share one understanding of the expected behavior.

Practical Example: Online Food Ordering App

Requirement: "Customers get a discount on large orders."

- **A tester reviewing it early asks:** What is "large"? Is the discount a percentage or a fixed amount? Does it combine with coupons? Does it apply before or after tax?
- **The team clarifies:** 10% off for orders of ₹1,000 or more, not combinable with coupons, calculated before tax.
- The tester immediately designs boundary cases (₹999 / ₹1,000 / ₹1,001) and a coupon-conflict case.

Without early involvement: developers guess the rule, the mismatch appears in system testing, and the code and tests are reworked. With early involvement: the ambiguity is removed in a meeting, so the defect never enters the code.

Changes in Team Collaboration Practices

- Include testers in requirement and design review meetings and sprint planning.
- Adopt a "three amigos" session (business analyst, developer, tester) before each story is built.
- Define acceptance criteria and Definition of Done jointly, with testing included.
- Use shift-left practices: unit tests, static analysis and peer review on every commit, with CI running automated checks.
- Share a common tool (JIRA/Confluence) for requirements, test cases and defects with traceability.
- Hold short daily stand-ups and a retrospective each cycle where defect trends and process gaps are discussed.
- Encourage mutual feedback: developers explain design decisions, and testers share defect patterns and risk areas.
- Keep testers' independence for objective evaluation while keeping communication open.

Conclusion

Bringing testers into the earliest phases turns testing from late defect detection into defect prevention. Clearer requirements, earlier test design and closer collaboration reduce rework, shorten delivery time and raise the quality of the final product.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Closing Coverage Gaps in an Inconsistently Failing System

Testing Principles Applied to Find the Gaps

Principle	Gap Revealed
Testing shows presence of defects	Passing cases prove nothing about untested inputs; failures in other cases show where coverage is missing
Exhaustive testing is impossible	Cases were picked by convenience, not by risk or input classes, so entire partitions were skipped
Pesticide paradox	The same "happy path" cases are rerun and no longer expose new defects
Defect clustering	Failures cluster around boundaries, special inputs and rarely used branches that were never tested
Early testing	Requirements were not analyzed for all conditions, so cases were never derived for them

Principle	Gap Revealed
Absence-of-errors fallacy	Correct output on a few cases was mistaken for a correct system

Gap-finding method:

build a requirement-to-test traceability matrix, then run a coverage tool. Requirements with no test case, and code lines or branches never executed, are the gaps.

How Proper Coverage Ensures Reliability

- Requirement coverage confirms every stated behavior has at least one test.
- Code coverage (statement, branch, condition) shows which logic has never run, and untested logic is where hidden defects stay.
- Input coverage (partitions, boundaries) ensures behavior is checked across the whole input space, not at a few points.
- Measurable coverage gives objective exit criteria, so release confidence rests on evidence.

Improving Test Case Design

Example: Discount rule: "orders $\geq$ ₹1,000 get 10% off; members get extra 5%."

Before (incomplete)	After (designed)
₹500 non-member, ₹2,000 member	BVA: 999 / 1,000 / 1,001
Only valid values	EP: negative, zero, text, very large amount
No rule combinations	Decision table: member $\times$ order $\geq$ 1,000 (4 combinations)

Result:

The earlier cases passed, but the designed set exposes that exactly ₹1,000 gets no discount ($>$ coded instead of $\geq$) and that member plus large order applies the wrong total.

Steps: list conditions from requirements $\rightarrow$ apply techniques $\rightarrow$ add negative and error cases $\rightarrow$ review with peers $\rightarrow$ map to requirements $\rightarrow$ check coverage $\rightarrow$ add cases for each new defect.

Techniques for Better Validation

Category	Techniques
Black-box	Equivalence partitioning, BVA, decision tables, state transition, use-case testing
White-box	Statement, branch and condition coverage; path testing
Experience-based	Error guessing, exploratory testing
Combinatorial	Pairwise testing to reduce parameter combinations
Quality aids	Peer reviews of test cases, mutation testing to check test strength, coverage tools (JaCoCo, Istanbul), traceability matrix
Regression	Automated suite rerun on every change

Conclusion:

Inconsistent behavior is a signal of coverage gaps, not randomness. Applying testing principles to locate untested requirements, inputs and code paths, and then designing cases with proven techniques and measured coverage, makes validation complete and the system dependably correct.

16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Improving Defect Reporting in a Logistics Application

Why Identification Is Delayed

Reports such as "tracking not working" give no steps, data or environment. Developers spend time asking questions and reproducing the issue, and the defect waits in the queue. Failures, defects and change requests also get mixed together, so nothing is triaged properly.

Clear Definitions Help Tracking

- **Error:** the human mistake, e.g., the developer misreads the delivery-slot rule.
- **Defect:** the flaw in code or documentation, e.g., the slot check uses < instead of <=.
- **Failure:** the visible wrong behavior, e.g., a 5:00 PM delivery is rejected.

Benefit of clear terms	Effect on tracking
Failure is separated from defect	Tester reports what was observed, developer locates the cause
Defect is separated from change request	Metrics count real faults, not new features
Root-cause (error) is recorded	Recurring causes such as unclear requirements become visible
Consistent status (new, assigned, fixed, retested, closed, reopened)	Everyone sees the same progress

Structured Reports Enhance Communication

Field	Poor Report	Structured Report
Title	"Shipment issue"	"Status stays 'In Transit' after delivery confirmation"
Steps	None	1. Open order #1045 2. Driver marks delivered 3. Refresh tracking page
Expected / Actual	Missing	Expected: "Delivered"; Actual: "In Transit"
Environment	Missing	Build 4.2, Chrome 120, Windows 11, staging

Field	Poor Report	Structured Report
Severity / Priority	Missing	Major / High (customers see wrong status)
Evidence	None	Screenshot, API response, log extract

Example:

Instead of "app shows wrong route," the structured report states the pickup and drop-off pincodes, the route returned, and the expected route. The developer reproduces it in minutes, finds the defect in the distance-calculation module, and fixes it the same day. The tester retests using the same steps, and the report becomes a regression case.

Improvements in Reporting Standards

- Adopt a standard defect template (aligned with IEEE 829/1044) with mandatory fields in JIRA or a similar tool.
- Define severity and priority scales with logistics examples (wrong delivery address = critical).
- Write one defect per report, with a neutral, factual tone and reproducible steps.
- Search for duplicates before logging, and link related defects.
- Add traceability to the requirement, test case and build.
- Set response-time targets by severity, with a regular triage meeting and workflow ownership.
- Track metrics such as defect aging, reopen rate and detection-to-fix time on a dashboard.
- Train testers and developers on the template, and review rejected reports to improve quality.

Conclusion:

Precise definitions let the team separate failure, defect and cause, while structured reports give developers everything needed to act at once. Standard templates, severity rules and tracked metrics shorten identification time and make defect communication reliable.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Achieving Consistent Testing Across Modules Through a Standard Process

Why Consistency Is Lost

Each module team tests in its own way: different templates, different entry and exit rules, different defect labels and different depth of coverage. Results cannot be compared, gaps go unnoticed, and quality depends on who tested.

Process-Based Approach

Define one organization-wide test process and apply it to every module:

Stage	Standard Activity	Common Output
Planning	Scope, risks, schedule, entry/exit criteria	Test plan (IEEE 829 format)
Design	Techniques chosen by rule (BVA, EP, decision table)	Test cases from one template
Setup	Controlled environment and test data	Environment checklist
Execution	Same run, logging and evidence rules	Test log, pass/fail status
Defect handling	Common severity scale and lifecycle	Defect reports in one tool
Closure	Exit criteria check, metrics, lessons learned	Test summary report

A maturity model such as TMMi or ISO/IEC/IEEE 29119 can guide how the process is defined, measured and improved.

How Standardized Processes Improve Quality

- **Repeatability:** any tester following the same steps produces comparable results.
- **Coverage visibility:** a common traceability matrix shows which module lacks tests.
- **Fair quality gates:** identical exit criteria prevent a weak module from being released on looser rules.
- **Reliable metrics:** defect density and removal efficiency are measured the same way, so trends can be trusted.
- **Easy onboarding and reuse:** templates, test data and suites are shared across modules.
- **Continuous improvement:** lessons from one module raise the standard for all.

Illustrative Example: Retail Application

Modules: Login, Cart, Payment, Reports.

- **Before:** Login testers use detailed cases; Cart testers use exploratory notes only; Payment defects are labelled "high/low" while Reports uses "P1-P4". Release readiness cannot be compared, and Cart defects escape.

- **After:** All modules use one template, and each must apply BVA and EP to inputs. Exit criteria are common: 100% critical cases pass, no open high-severity defect, and at least 85% branch coverage. Defects use one severity scale in JIRA.
- **Result:** The Cart module's untested coupon boundaries become visible in the traceability matrix, and the missing cases are added before release. Module reports can now be compared on the same metrics.

Methods to Ensure Consistency

- Publish a test process document with templates, checklists and a defect lifecycle.
- Use one tool set (JIRA for defects, a test management tool for cases, Jenkins for regression).
- Apply peer reviews and audits of test plans and cases against the standard.
- Set common exit criteria and quality gates in CI.
- Maintain a shared regression suite and a reusable test data repository.
- Track uniform metrics (coverage, defect density, defect removal efficiency, reopen rate) on a dashboard.
- Nominate a test lead or test process group to own the standard, train teams and review compliance.
- Hold retrospectives each release to update the process from observed defects.

Conclusion:

A defined, measured test process removes dependence on individuals and gives every module the same depth, rules and reporting. The team gets comparable results, fewer escaped defects and a steadily improving quality baseline.

18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Edge Cases Missed Before Deployment

Testing Principles: Why the Edge Cases Were Missed

Principle	Reason Edge Cases Escaped
Exhaustive testing is impossible	Cases were picked by convenience, so typical inputs were covered and extremes were never selected
Defect clustering	Defects gather at limits, empty inputs and rare combinations, exactly where testing was thin
Pesticide paradox	The same happy-path suite was reused each release and stopped revealing new defects
Early testing	Limits, formats and error conditions were never clarified in requirements, so no cases were derived

Principle	Reason Edge Cases Escaped
Absence-of-errors fallacy	Passing normal cases was mistaken for a correct system
Context dependence	Production brings real data volumes, unusual users and odd sequences that test data never reflected

How Boundary Testing Helps

Defects cluster at the edges of input ranges because developers commonly make off-by-one mistakes ($<$ vs $<=$). Boundary Value Analysis (BVA) tests the values at, just below and just above each limit, so these faults are exposed with very few test cases.

Example: "Password length must be 8 to 16 characters."

Value	Length	Expected
Below minimum	7	Reject
Minimum	8	Accept
Maximum	16	Accept
Above maximum	17	Reject

If the code uses < 16 , only the length-16 case fails. Random lengths such as 10 or 12 would never find it.

Including Edge Cases in Testing

1. **List conditions from each requirement:** ranges, sizes, formats, counts, dates, and time limits.
2. Apply BVA and equivalence partitioning to each condition.
3. **Add special values:** null, empty string, zero, negative, maximum integer, very long text, special characters, Unicode.
4. **Add situational edges:** leap day (29 Feb), month-end, time zone change, daylight saving, midnight rollover, an empty list or one item, a full database, duplicate submissions.
5. **Add sequence and state edges:** an expired session, a repeat click, a network drop mid-operation, a concurrent update.
6. Record expected results before execution, and trace each case to its requirement.
7. Convert every production failure into a permanent regression case.

Improvements in Test Design

- Checklists of edge conditions reused across modules and updated after each incident.
- Decision tables and state transition testing for rule combinations and unusual sequences.
- Pairwise testing to cover parameter combinations without a huge suite.
- Exploratory sessions with error guessing, time-boxed and driven by past defects.
- Data-driven automation to run many boundary values cheaply on every build.
- Peer review of test cases and traceability matrices to reveal missing conditions.
- Production-like test data and monitoring, with logs feeding new edge cases into the suite.
- Coverage measurement (branch and condition) to find code paths no test has reached.

Conclusion:

Edge-case failures come from tests chosen by convenience rather than by design. Applying testing principles explains the gap, and boundary value analysis plus structured edge-case lists, strong test design and automation catch these defects before deployment, giving a more dependable release.

19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Fixing Defect Documentation with a Structured Repository

The Problem

Defects live in emails, chats and spreadsheets, with missing steps, no owner and no status. Issues get lost, the same bug is reported twice, and nobody can say how many defects are open, who owns them, or why they keep appearing.

Defect Repository as the Solution

A defect repository is one central, searchable record of every defect from detection to closure, and each entry holds fixed fields:

Field Group	Details Recorded
Identification	Unique ID, title, reporter, date, module, build
Description	Steps to reproduce, expected vs actual result, environment, evidence (screenshot, log)
Classification	Type, severity, priority, phase injected and detected
Lifecycle	Status (new, assigned, fixed, retested, closed, reopened), owner, fix version
Analysis	Root cause, fix description, linked requirement and test case

How Documentation Improves Defect Management

- **Traceability:** every defect can be followed from requirement to test case to fix to retest.
- **Accountability:** a named owner and status show who must act next.
- **Faster resolution:** complete steps and evidence let developers reproduce the failure at once.
- **Fewer duplicates and lost issues:** one source of truth that can be searched.
- **Informed decisions:** managers see open counts by severity and judge release readiness.
- **Organizational memory:** past fixes guide future debugging, and new team members learn quickly.

How Structured Repositories Support Analysis

Example: Banking app, last three releases, 48 defects logged.

- Grouping by module shows 20 defects in Fund Transfer (defect clustering).
- Grouping by type shows 14 of them are boundary and validation defects.
- Grouping by phase injected shows most came from unclear limit rules in requirements.
- Trend charts show the count falling after the team clarified requirements and added boundary tests.

None of this is possible from scattered emails. With structured data, Pareto charts, trend graphs and root-cause analysis show where to act: clarify the requirement, add boundary cases to regression, and add a review checklist item.

Improvements in Documentation Practices

- Adopt a standard defect template (aligned with IEEE 829/1044) with mandatory fields in JIRA or Bugzilla.
- Define severity, priority and status scales with examples so everyone uses them alike.
- Log one defect per report, in factual language, with reproducible steps and evidence.
- Record root cause and fix details before closing a defect.
- Search for duplicates before raising a new report, and link related defects.
- Link each defect to its requirement, test case and build.
- Hold regular defect triage and review meetings to check quality of entries and aging defects.
- Track metrics such as defect density, aging, reopen rate and defect removal efficiency on dashboards.
- Train the team on the template and audit entries periodically.

Conclusion:

A structured defect repository turns scattered complaints into reliable, traceable data. Consistent documentation speeds fixes, prevents lost or repeated issues, and supports pattern analysis, so the company can manage defects and prevent them in later releases.

20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Redesigning Testing for Reliable Transaction Processing

Testing Principles Applied to the Redesign

Principle	Redesign Action
Testing shows presence of defects	Treat each failed transaction (duplicate debit, wrong balance) as evidence, then trace it to a defect and its cause
Exhaustive testing is impossible	Rank transaction types by risk: transfers, bill payments, standing orders, reversals
Early testing	Review limits, fees, cut-off times and reversal rules at the requirements stage
Defect clustering	Focus on hot spots such as concurrency, reversals and interfaces to payment gateways
Pesticide paradox	Review and refresh the suite each release, adding cases from every production failure
Context dependence	Banking needs strict accuracy, audit trails, security and compliance checks
Absence-of-errors fallacy	Correct individual screens do not prove correct end-to-end money movement

How Structured Testing Improves Quality

- **Planned coverage:** each requirement maps to test cases, so gaps are visible before release.
- **Repeatability:** the same documented cases give the same verdict whoever runs them.
- **Early detection:** reviews and unit tests catch defects when fixing is cheapest.
- **Control:** entry/exit criteria (for example, no open critical defects) stop unready builds.
- **Learning:** logged and classified defects reveal root causes and prevent repeats.

Illustrative Examples

Failure	Defect	Test That Exposes It
Customer debited twice when the network drops and the request is retried	No idempotency check	Retry and timeout test; assert exactly one debit
Transfer of ₹1,00,000 accepted although the limit is ₹1,00,000 per day, then ₹1 more also accepted	Wrong comparison (> vs >=)	BVA at 99,999 / 1,00,000 / 1,00,001
Two simultaneous withdrawals both succeed on a balance enough for one	Race condition	Concurrency test on the same account

Failure	Defect	Test That Exposes It
Failed credit leaves the debit posted	Missing rollback	Transaction (ACID) failure-injection test
Interest differs by a paisa	Rounding error	Data-driven calculation test against a trusted oracle

Techniques to Enhance Defect Detection

- Boundary value analysis and equivalence partitioning for amounts, limits and account types.
- Decision tables for fee, limit and eligibility rules.
- State transition testing for account and transaction states (pending, posted, reversed, failed).
- Concurrency and stress testing with JMeter for peak loads such as salary day.
- Failure injection and recovery tests for timeouts, partial updates and system restarts.
- Reconciliation testing: total debits must equal total credits after batch runs.
- Security tests: authorization, session expiry and tamper checks.
- Automated regression in CI, so every build reruns the critical transaction suite.
- Risk-based prioritization, code reviews and mutation testing to measure test strength.
- Defect repository analysis to feed past defect patterns back into new test cases.

Conclusion:

Applying testing principles replaces ad-hoc checks with risk-based, requirement-traced and repeatable testing. Techniques such as BVA, concurrency, rollback and reconciliation tests find the defects that cause transaction failures before release, giving a banking system that is accurate, consistent and trusted.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand